

Creating your first SE Mod

A follow along tutorial for creating your first Script Extender Mod that stops companions from returning to their tent when in camp. Optional toggleable version

Creating your first SE Mod



Preface: This page is a work in progress!

This tutorial will walk you through creating a simple mod with Norbyte's Script Extender(SE) that stops your companions from going back to their tent while in camp. Additionally we will utilize SE together with [stats](#) to make this behaviour toggleable.

This tutorial is based on **Satan & Alithea Ancunin's** implementation of [Stay still in camp and play idle animations anywhere](#) [↗](#). Please note that the final mod also adds idle animations, which isn't part of this tutorial.



If you have not used SE before, it's recommended to read [Getting started with Script Extender](#) [first](#).

1. Outlining your strategy

First, you are going to be organizing the functions and game elements you will be needing for your mod. One of the best resources for this can be found on [Laughing Leader's Github](#) [↗](#), which contains a list of Osiris (native) functions you can use.

To find the flags, spells, passives or any other game elements you might need, you can use [Norbsearch](#) [↗](#), or index your game files using [ShinyHobo's Multitool](#) [↗](#).

2. Preparing your workspace

Once you've outlined the necessary functions, flags, spells, items, and other elements, you need to prepare your workspace. It's important to create a proper file structure:

Replace MODNAME with the name of your mod.

```
MODNAME/  
  Mods/  
    MODNAME/  
      meta.lsx  
      ScriptExtender/  
        Config.json  
        Lua/  
          BootstrapServer.lua  
          Server/  
            MODNAME.lua
```

Upon organizing your folder structure, you can go ahead and pack the mod, which - if you're using BG3 Multitool - will prompt you to add an author (That's your name!), a description for your mod and a version.

You are now ready to begin coding your mod!



An [example workspace](#)  to adjust accordingly can be viewed and downloaded from google drive.

3. Writing the code

3.1 Defining your Constants

First, we will define our constants. These are variables that will be used throughout our code. They are usually used for flags, UUIDs, or any important value you would like to use in more than one function/event.

In the following example, we are defining two constants, FLAG_IN_CAMP, which is defined as the UUID of an immobile character, and ModAuthor, which is defined as a simple string with the value "Smart".

```
1 | -- variables
2 |
3 | FLAG_IN_CAMP = "161b7223-039d-4ebe-986f-1dcd9a66733f"
4 | ModAuthor = "Smart"
```

3.2 Writing your Methods

We will now use our previously defined variables in functions (methods) of our design. In the following example, we are creating two functions, stopMoving and startMoving:

```
1 | -- methods
2 |
3 | ---@param character GUIDSTRING      - the character who will
4 | stop moving in camp
5 | function stopMoving(character)
6 |     if Osi.GetFlag(FLAG_IN_CAMP, character) == 1 then
7 |         Osi.ClearFlag(FLAG_IN_CAMP, character)
8 |     end
9 | end
```

```
1 | ---@param character GUIDSTRING      - the character who will
2 | start moving again in camp
3 | function startMoving(character)
4 |     if Osi.GetFlag(FLAG_IN_CAMP, character) == 0 then
5 |         Osi.SetFlag(FLAG_IN_CAMP, character)
6 |     end
7 | end
```

3.3 Making your Code Event-driven

Mods are often, if not always, driven by Game Events, moments in the game that trigger an “event”, which we will “listen” to and apply our functions afterwards.

For example, if we wanted to apply the stopMoving function after a character enters combat, this is how it would look like:

```
1 | Ext.Osiris.RegisterListener("EnteredCombat", 2, "after",
2 | function(character, combatid)
3 |     stopMoving(character)
   | end)
```

You can find more events at

<https://github.com/LaughingLeader/BG3ModdingTools/blob/master/generated/Osi.Events.lua>



4. Interdiscipline: Using stats file in conjunction with SE

You can call game statuses, passives or other elements inside your SE functions and events. You can also create your own statuses and passives. Here are examples of a status and a passive.

4.1 Status

```
1 | new entry "STAY_STILL_STATUS"
2 | type "StatusData"
3 | data "StatusType" "BOOST"
4 | data "DisplayName" "h38fc0213b20a4412bf0f223aeda5a7e974a9"
5 | data "Description" "hd5dfdb8b7aa34a808f628362b0f04fa8884a"
6 | data "Icon" "Spell_Conjuration_DimensionDoor"
7 | data "StackId" "STAY_STILL_STATUS"
```

4.2 Passive

```
1 | new entry "STAY_STILL_PASSIVE"
2 | type "PassiveData"
3 | data "DisplayName" "h03652d052f114f8ebbc10bc2602cddc835e7"
4 | data "Description" "h31d5cb6c31ed4d198231d44624caf7c59f5b"
5 | data "Icon" "Spell_Conjuration_DimensionDoor"
6 | data "Properties" "IsToggled;ToggledDefaultAddToHotbar"
7 | data "ToggleOnFuncutors"
8 | "ApplyStatus(STAY_STILL_STATUS,100,-1)"
   | data "ToggleOffFuncutors" "RemoveStatus(STAY_STILL_STATUS)"
```

In order to display tooltips and/or descriptions in-game, you will need to create a Localization file with the extension .loca.xml. It is highly recommended to use the BG3 Mod Helper VSCode extension for this purpose, as it allows you generate handles inside your stats file and automatically creates localizations for you.

Below is an example of a Localization file. As you can see, the contentuids correspond to the descriptions and display names of the Status and Passive above.

4.3 Localization File

```

1  <?xml version="1.0" encoding="utf-8"?>
2  <contentList>
3
4      <!-- Toggle for entity to stay still in camp -->
5
6      <content
7  contentuid="h03652d052f114f8ebbc10bc2602cddc835e7"
8  version="1">Stay still in Camp</content>
9      <content
10 contentuid="h31d5cb6c31ed4d198231d44624caf7c59f5b"
11 version="1">When this is active, the companion will not move
12 back to their tent while in camp</content>
13
14      <!-- Passive Applied to entity when it is standing still -
15      -->
16
17      <content
18  contentuid="h38fc0213b20a4412bf0f223aeda5a7e974a9"
19  version="1">Standing still</content>
20      <content
21  contentuid="hd5dfdb8b7aa34a808f628362b0f04fa8884a"
22  version="1">Entity will not move back to Camp</content>
23  </contentList>

```

5. Using SE to Handle the Stats

As we've mentioned above, you can use SE to handle and call stats in your code. If you wanted to add the previously defined passive to your party, here is how that would look like in your script:

5.1 Adding the Passives to the Party

We will first create a function that adds a passive to a character that doesn't have that passive.

```

1  ---@param character GUIDSTRING      - the character receiving
2  the passive
3  ---@return passive - string          - the passive being added
4  to the character
5  function addPassive(character, passive)
6      if Osi.HasPassive(character, passive) == 0 then
7          Osi.AddPassive(character, passive)
8      end
9  end

```

We will use the LevelGameplayStarted event (After a save is loaded) to add the "Stay in Camp" toggle passive to each party member.

```

1  -- Adds the "Stay in Camp" toggle to each party member
2  Ext.Osiriris.RegisterListener("LevelGameplayStarted", 2,

```

```

3  "after", function(_, _)
4      _P("LevelGameplayStarted")
5
6      local party = Osi.DB_PartyMembers:Get(nil)
7      for i = #party, 1, -1 do
8          addPassive(party[i][1], "STAY_STILL_PASSIVE")
9      end
10
      end)

```

We will also add that toggle passive to any party member that joins the party during gameplay, using the event "CharacterJoinedParty".

```

1  -- Adds the "Stay in Camp" toggle to a partymember added
2  during gameplay
3  Ext.Osiris.RegisterListener("CharacterJoinedParty", 1,
4  "after", function(character)
5      addPassive(character, "STAY_STILL_PASSIVE")
6  end)

```

5.2 Responding to the Passive being Toggled On

When the passive is toggled on, the "TeleportedToCamp" event is triggered. If the STAY_STILL_STATUS status is on, we will call the "stopMoving" function and stop the character from moving.

```

1  -- Stops the partymember from moving if "Stay Still" is
2  activated and they are teleported to camp
3  Ext.Osiris.RegisterListener("TeleportedToCamp", 1, "after",
4  function(character)
5      _P("Teleported to camp")
6      if Osi.HasPassive(character, "STAY_STILL_PASSIVE") == 1
7  then
8          _P("character has ", "STAY_STILL_PASSIVE")
9          stopMoving(character)
10     end
11 end)

```

```

1  -- Stops the partymember from moving if "Stay Still" is
2  activated and they are already in camp
3  Ext.Osiris.RegisterListener("StatusApplied", 4, "after",
4  function(character, status, _, _)
5      if status == "STAY_STILL_STATUS" then
6          stopMoving(character)
7      end
8  end)

```

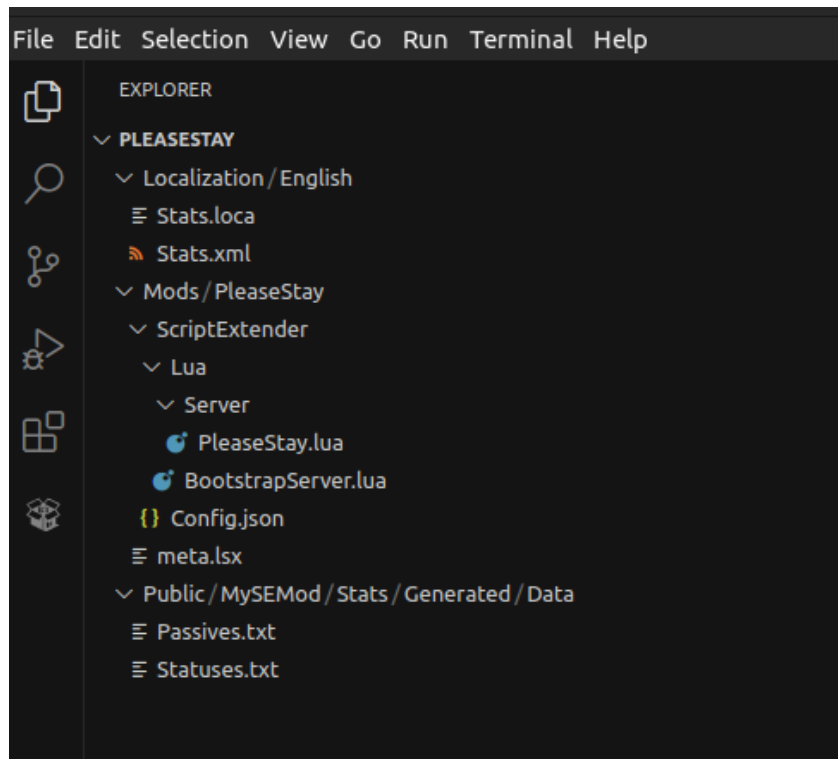
5.3 Responding to the Passive being Toggled Off

When the passive is toggled off, another event is triggered, the "StatusRemoved" event, where the Status in question is "STAY_STILL_STATUS". If that status is removed, we will call the "startMoving" function and allow the character to move again.

```

1 | -- Allows the partymember to move again if "Stay Still" is
2 | deactivated
3 | Ext.Osiris.RegisterListener("StatusRemoved", 4, "after",
4 | function (character, status, _, _)
5 |     if status == "STAY_STILL_STATUS" then
6 |         startMoving(character)
7 |     end
8 | end)

```



WIP

to be continued...

Credits:

Alithea Ancunin for the idea. Without her this would not exist.

Powered by [Wiki.js](#)